Engineering Reliable Agentic AI Systems

Session 1. System Architecture, the foundation.

Saturday 29 August 2026. 10:00 Central.

Four artifacts. You leave with all four.

- A running autonomous loop, in the first hour
- A reusable evaluation harness
- One live research assistant over Model Context Protocol (MCP)
- A production architecture you can hand to your team

Prompting dies under volume.

- One clever prompt works once.
- Ten tickets a day, it drifts.
- A hundred, and nobody remembers what good looked like.
- The bottleneck is not the model. It is you, reading every diff.

A loop is not "call the model until it says done."

A production loop is a state machine. Every iteration:

1. starts from **explicit state**, not from chat history,
2. gets **bounded authority**, not the whole repo,
3. produces **observable evidence**, not a claim,
4. and passes through a transition function **you** enforce, not the model.

Take away any one of the four and you have a generator with a while loop.

The evidence for looping is not vibes.

AlphaCodium, on the CodeContests validation set:

Approach	pass@5
One well-designed direct prompt	19%
Plan, generate against tests, iterate	44%

Same model. The flow did that, not the prompt.

Ridnik et al., arXiv:2401.08500

The object is a CRM, and it lives in another repo.

- A small customer relationship management app. Customers. Sales tasks.
- Not a ticketing app. Too meta.
- The engine never imports it. You point the loop at a path.
- First ticket: add a due date. Vague on purpose.

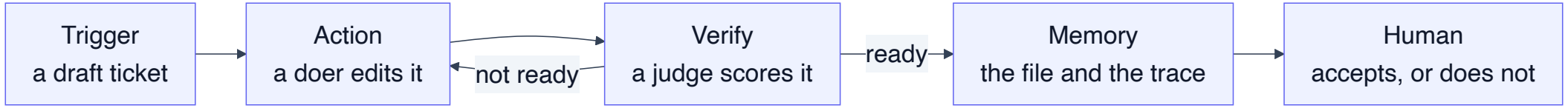
The clock, and permission to fall behind.

- 10 minutes open. 45 this module. Then a break.
- The lab is 25 minutes of typing, not 45.
- Stuck? Stop typing and watch. `git checkout done-m1` and you continue with a working artifact.

Nobody leaves this room behind.

Anatomy of an agent loop

Trigger. Action. Verify. Memory. Human oversight.



Five parts. **Verify** is the one that separates a loop from a script that calls a model.

Trigger

- Something outside the model starts the work.
- Ours today: a draft markdown ticket in the target repo.
- Not a chat. Not "hey, add due dates."
- In production it is a webhook or a schedule, and it fires only when the branch head actually moved. A trigger that fires on no change burns budget.

Action, inside a scope you declared

- A **doer** writes files. Only inside a declared scope.
- The scope lives in `.loop.yml`, in the target repo.
- It is enforced at the tool boundary, not in the prompt.

An agent can argue its way past an instruction. It cannot argue its way past a tool it was never given.

Verify

- A **judge** scores the result. It reports, and that is all it does.
- The judge holds no write path. Not a rule. A missing method.
- Today it answers: is this ticket a contract a test could fail?

If verify is "looks good to me," you do not have a loop. You have a generator.

Memory, and why the context window is not it

Liu et al. 2024 measured it. Accuracy is highest when the fact sits at the **start** or the **end** of the context. Move it to the middle and accuracy drops by more than 30%.

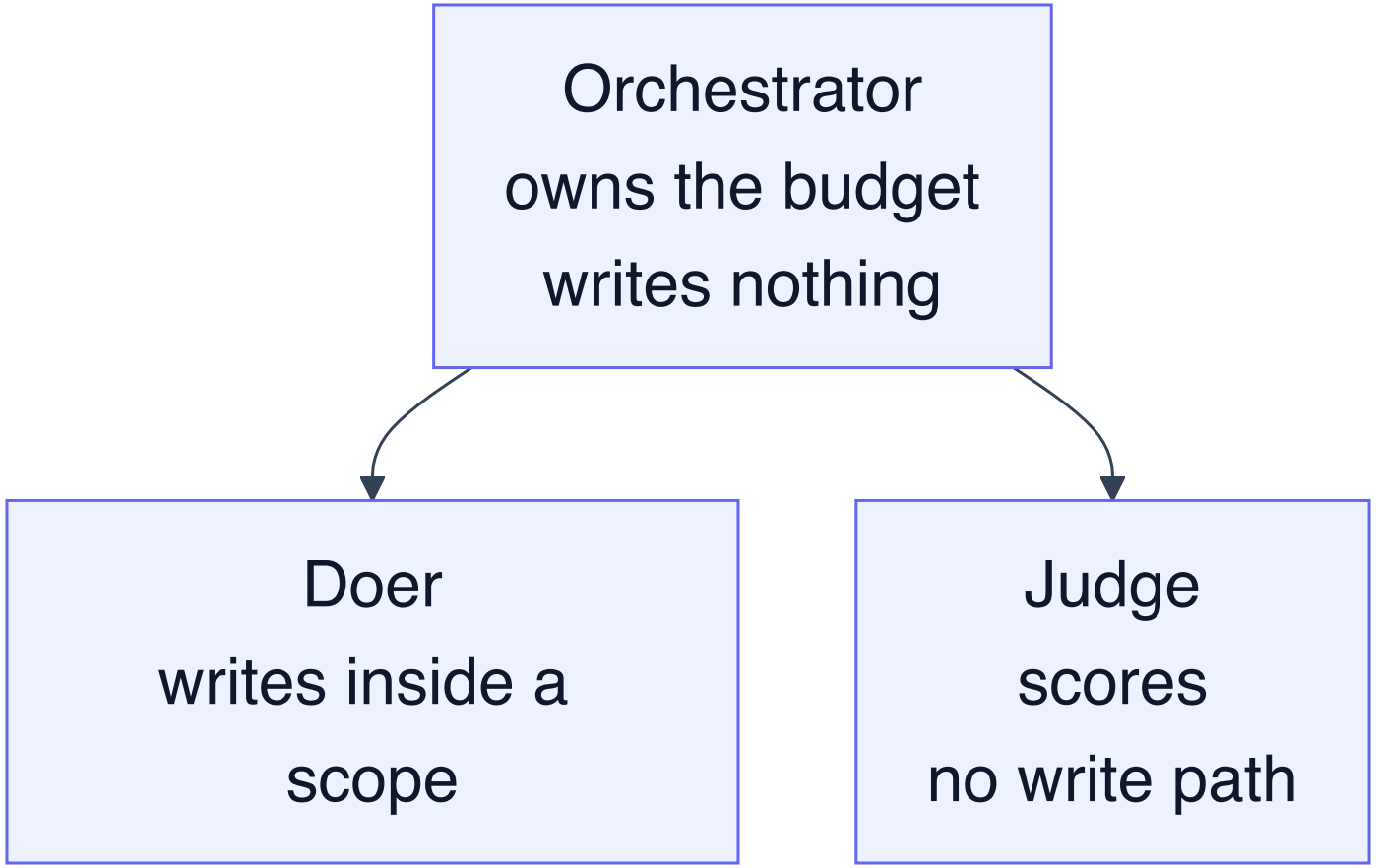
- Reproduced on GPT-4, Claude, MPT-30B, and Cohere Command.
- So state goes on disk: the ticket, the trace, the plan.
- Big output goes to a file. Only a short summary returns to the orchestrator.

Human oversight

- The loop proposes. A human accepts.
- Today: the loop rewrites the ticket. You decide it is a contract.
- In Module 2 the loop opens a pull request. It still does not merge.

Oversight is a designed step, not a hope.

Three parts, and the object is the only variable.



Module	Object

Lab 1. The Ticket Enhancer. 25 minutes.

```
cd labs/m1-enhancer
claude -p "$(cat prompts/claude-code.md)"     # or codex, grok, opencode

task loop:enhancer -- --ticket T001           # it escalates. that is correct.
task loop:enhancer -- --ticket T001 --incorporate
```

Fill `loop.py`. Two functions: `judge_ticket` and `decide_next`.

This lab writes no code, so the push gate stays quiet. You meet it in Module 2.

Falling behind is fine: `git checkout done-m1`.

Read the trace. The interesting run is the one that stops.

```
round 1: feature, not ready
  missing: why it is worth doing
  missing: acceptance criteria a test can fail
round 2: feature, not ready
  missing: why it is worth doing
  missing: acceptance criteria a test can fail

gate: escalate
reason: the same rows failed twice. The loop is not converging.
```

An iteration that burns tokens and reproduces the identical failure is not progress. Stopping is the feature.

Where this breaks at scale

- **No contract.** The doer invents the field type, and you find out in review.
- **No stop.** It edits forever, and the bill arrives on Monday.
- **No scope.** It weakens the test instead of fixing the code.
- **Context rot.** The whole repo goes into the window and quality falls.

Each one has a fix. All four fixes are Module 2.

Break. 15 minutes.

Next: the harness. Two doers, a red gate, ten rubric rows, and a gate that refuses to let you push.

Module 2 is the one that does not get cut.